

DevOps Engineer

Interview Questions & Answers

Jenkins · Docker · Kubernetes · Terraform · AWS

SECTION 1 — Jenkins & CI/CD

Q1. Which type of Jenkins pipeline do you prefer — Declarative or Scripted? Why?

ANSWER

- **Declarative Pipeline** is the preferred choice for most teams.
 - Uses a structured, opinionated syntax defined inside a **pipeline { }** block — easier to read and maintain.
 - Built-in validation catches syntax errors before execution.
 - Supports **post**, **options**, and **environment** blocks natively — less boilerplate.
 - **Scripted Pipeline** (Groovy DSL) offers full programmatic control but is harder to maintain and debug at scale.
 - Use Scripted only when Declarative cannot express the logic — e.g., dynamic parallel stages based on runtime data.
-

Q2. Can you write and explain a Jenkins pipeline?

ANSWER

- A Declarative pipeline has four key sections: **agent**, **environment**, **stages**, and **post**.
- **agent any** — runs on any available Jenkins node.
- **environment** — defines variables available across all stages.
- **stages** — sequential steps: Checkout → Build → Test → Docker Build → Deploy.
- **post** — runs always/on failure/on success for notifications and cleanup.

```
pipeline {
  agent any
  environment { IMAGE = 'myapp:latest' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'mvn clean package' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Docker') { steps { sh 'docker build -t $IMAGE .' } }
    stage('Deploy') { steps { sh 'kubectl apply -f k8s/' } }
  }
  post {
    failure { slackSend message: 'Build failed!' }
    success { echo 'Pipeline succeeded.' }
  }
}
```

Q3. What stages have you used in your CI/CD pipeline in your project?

ANSWER

- **Checkout** — Pull source code from Git/GitHub.
- **Build** — Compile using Maven / Gradle / npm.
- **Unit Test** — Run test suites; publish JUnit reports.
- **Code Quality** — SonarQube static analysis gate.
- **Docker Build & Push** — Build image, tag with commit SHA, push to ECR/DockerHub.
- **Deploy to Staging** — Apply Kubernetes manifests or Helm charts to staging namespace.
- **Smoke Test** — Basic health-check against staging URL.
- **Deploy to Production** — Manual approval gate → rolling update to prod.
- **Notify** — Slack/email on success or failure.

Q4. Where do you store secrets in Jenkins?

ANSWER

- **Jenkins Credentials Store** — built-in; supports Username/Password, Secret Text, SSH keys, Certificates.
- Reference in pipeline: `withCredentials([string(credentialsId: 'my-token', variable: 'TOKEN')])`.
- **HashiCorp Vault Plugin** — preferred for enterprise; secrets fetched at runtime, never stored in Jenkins.
- **AWS Secrets Manager / Parameter Store** — accessed via AWS CLI or SDK inside pipeline steps.
- Never hardcode secrets in Jenkinsfile or pass them as plain environment variables that appear in logs.
- Use **mask-passwords** plugin to prevent accidental secret exposure in console output.

Q5. If your Jenkins server is not accessible, how would you troubleshoot it?

ANSWER

1. **Check process** — SSH to server; verify Jenkins is running: `systemctl status jenkins`.
2. **Check port** — Confirm port 8080 is open: `netstat -tlnp | grep 8080`.
3. **Firewall/Security Groups** — Ensure inbound rules allow the port (especially in AWS).
4. **Jenkins logs** — Review `/var/log/jenkins/jenkins.log` for startup errors or OOM kills.
5. **Disk space** — Full disk prevents Jenkins from starting: `df -h`.
6. **Java/JVM** — Jenkins requires Java; check version and heap: `java -version`.
7. **Restart** — `systemctl restart jenkins`; monitor logs post-restart.
8. **Nginx/Reverse Proxy** — If behind a proxy, check proxy config and SSL certificates.

Q6. Your Jenkins pipeline failed — how do you debug it?

ANSWER

1. **Console Output** — First stop; scroll to the red error line to identify the failing stage.
2. **Stage View** — Blue Ocean or classic UI shows which stage failed and its duration.
3. **Reproduce locally** — Run the failing shell command on the agent node manually.
4. **Environment variables** — Print `env` at the failing step to check for missing variables.
5. **Agent issues** — Verify the agent node is online and has required tools installed.
6. **Dependency/network** — Check if artifact registries or external APIs are reachable from the agent.
7. **Add `sh(returnStatus: true)`** — Capture exit codes for granular debugging.
8. **Replay Pipeline** — Jenkins' Replay feature lets you tweak and re-run without committing.

SECTION 2 — Docker

Q7. How do you reduce the size of a Docker image?

ANSWER

- **Use minimal base images** — prefer **alpine** or **distroless** over full OS images (e.g., `python:3.11-alpine` vs `python:3.11`).
- **Multi-stage builds** — compile in a builder stage, copy only the binary/artifact to a slim final stage.
- **Combine RUN commands** — chain with **&&** and clean up in the same layer: **`RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*`**.
- **.dockerignore** — exclude **node_modules**, **.git**, test files, and docs from the build context.
- **Avoid installing dev dependencies** in production images (e.g., **`npm install --production`**).
- **Use specific tags** — pin base image versions to avoid pulling unexpectedly large updates.
- **Remove temporary files** in the same RUN layer where they were created.

Q9. What is a Docker container?

ANSWER

- A container is a **lightweight, isolated runtime environment** that packages an application with all its dependencies.
- Built from a Docker **image** — a read-only template of filesystem layers.
- Uses Linux kernel features — **namespaces** (isolation) and **cgroups** (resource limits) — not a hypervisor.
- Shares the host OS kernel; much faster to start and uses less memory than a VM.
- Stateless by default — data written inside a container is lost when the container stops unless a **volume** is mounted.
- Key difference from a VM: containers virtualise the **OS**, VMs virtualise the **hardware**.

Q10. How do you check if a container is running?

ANSWER

- **docker ps** — lists all running containers with ID, image, status, ports, and name.
- **docker ps -a** — includes stopped/exited containers.
- **docker inspect <container_id>** — returns full JSON config including **State.Status** and **State.Health**.
- **docker stats** — live CPU, memory, and network metrics for running containers.
- **docker logs <container_id>** — check stdout/stderr for crash reasons if status is 'Exited'.
- In Kubernetes: **kubectl get pods** and **kubectl describe pod <name>** to check container readiness.

Q1
6.

What is the difference between a virtual machine and a container?

ANSWER

- **VM**: runs a full OS including its own kernel on top of a **hypervisor** (VMware, KVM, Hyper-V).
- **Container**: shares the host OS kernel; only packages the app and its libraries.
- **Size**: VMs are gigabytes; containers are megabytes.
- **Startup**: VMs take minutes to boot; containers start in seconds or milliseconds.
- **Isolation**: VMs provide stronger security isolation; containers rely on namespace/cgroup separation.
- **Use case**: VMs for strong multi-tenant isolation or running different OS types; containers for microservices, CI/CD, and scalable deployments.
- **Portability**: containers are more portable across environments via Docker images.

SECTION 3 — Terraform & AWS

Q8. Why do we use Terraform?

ANSWER

- Terraform is an **Infrastructure as Code (IaC)** tool that provisions and manages cloud resources declaratively.
- **Multi-cloud** — works with AWS, Azure, GCP, and 1000+ providers using the same workflow.
- **State management** — tracks real infrastructure state in a **.tfstate** file, enabling drift detection.
- **Plan before apply** — **terraform plan** shows exactly what will change before anything is touched.
- **Idempotent** — running the same config multiple times produces the same result; safe to re-apply.
- **Modules** — reusable, versioned infrastructure components reduce duplication.
- Alternative to CloudFormation (AWS-only) — Terraform is cloud-agnostic and has a larger community.

Q1
2.

What is the difference between an IAM Role and an IAM User?

ANSWER

- **IAM User**: a permanent identity (person or application) with long-term credentials — access key + secret key.
- **IAM Role**: a temporary identity with no long-term credentials; assumed by trusted entities for a defined session.
- Roles use **STS (Security Token Service)** to issue short-lived tokens — far more secure.
- **Best practice**: EC2 instances, Lambda functions, and ECS tasks should use **IAM Roles**, never access keys.
- Roles can be assumed cross-account; users cannot span accounts without federation.
- Auditing: Role usage is tracked via CloudTrail AssumeRole events; user key leaks are harder to detect.

Q1
3.

What is an S3 Bucket?

ANSWER

- S3 (Simple Storage Service) is AWS's **object storage** service — infinitely scalable, 99.999999999% durability.
 - A **bucket** is a top-level container for objects (files + metadata), unique globally across all AWS accounts.
 - Objects are addressed by **bucket name + key**; no true directory structure (prefix-based).
 - Common uses: static website hosting, Terraform state backend, CI/CD artifact storage, data lake, log archiving.
 - **Versioning** — retain multiple versions of an object; protects against accidental deletion.
 - **Bucket Policies & ACLs** — control access at bucket and object level.
 - **Encryption** — SSE-S3, SSE-KMS, or SSE-C for data at rest; TLS enforced in transit.
 - Storage classes: Standard, Intelligent-Tiering, Glacier — trade cost vs access latency.
-

SECTION 4 — Kubernetes

Q1
1.

How do you deploy an application to Kubernetes using a pipeline?

ANSWER

- **Step 1 — Build & push image:** Jenkins/GitLab CI builds Docker image, tags with Git SHA, pushes to ECR or DockerHub.
 - **Step 2 — Update manifest:** Replace image tag in **deployment.yaml** using **sed** or a Helm values override.
 - **Step 3 — Apply:** Run **kubectl apply -f k8s/** or **helm upgrade --install** from pipeline.
 - **Step 4 — Verify rollout:** **kubectl rollout status deployment/myapp** — blocks until pods are ready or times out.
 - **Step 5 — Rollback on failure:** **kubectl rollout undo deployment/myapp** if smoke tests fail.
 - Use **kubeconfig** stored as a Jenkins credential; never embed cluster credentials in the Jenkinsfile.
 - For GitOps: push manifest changes to Git; ArgoCD or Flux auto-syncs to the cluster.
-

Q1
4.

Can you write a Kubernetes Deployment YAML file?

ANSWER

- **apiVersion**: apps/v1 — Deployment is a stable API since Kubernetes 1.9.
- **replicas**: 3 — runs 3 pod instances; scheduler distributes them across nodes.
- **selector.matchLabels**: must match **template.metadata.labels** — links Deployment to its Pods.
- **resources**: always set **requests** (scheduling) and **limits** (throttling/OOM kill prevention).
- **livenessProbe**: Kubernetes restarts container if it fails; **readinessProbe**: removes from Service endpoints until ready.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myrepo/myapp:v1.2.3
          ports:
            - containerPort: 8080
          resources:
            requests: { cpu: '100m', memory: '128Mi' }
            limits: { cpu: '500m', memory: '512Mi' }
          livenessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 10
          readinessProbe:
            httpGet: { path: /ready, port: 8080 }
            initialDelaySeconds: 5
```

**Q1
5.**

If your application is not accessible, how would you troubleshoot it?

ANSWER

- **1. Check pods:** `kubectl get pods -n <ns>` — are pods Running or CrashLoopBackOff?
- **2. Pod logs:** `kubectl logs <pod> -n <ns> --previous` — check for application errors.
- **3. Describe pod:** `kubectl describe pod <pod>` — events show image pull failures, OOM kills, scheduling issues.
- **4. Service:** `kubectl get svc` — verify selector matches pod labels and port mapping is correct.
- **5. Endpoints:** `kubectl get endpoints <svc>` — empty endpoints means no pods match the selector.
- **6. Ingress/Load Balancer:** check Ingress rules, ALB target group health, and DNS resolution.
- **7. Network Policies:** ensure no policy is blocking traffic between namespaces or from the ingress.
- **8. AWS layer:** verify Security Groups, NACLs, and Route Tables if running on EKS.
- **9. Resource constraints:** `kubectl top pods` — OOM or CPU throttling can cause 502/503 errors.

All answers reflect industry best practices as of 2025. Adapt examples to your specific stack and project context.